

Programación Orientada a Objetos

(POO)

2026 – 2

Actividad N°2

Sharik Mileth Cuero Sinisterra

Grupo 2

Ejercicio 2.1 (PAG 63)

Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellido, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus respectivos atributos.

La clase debe incluir los siguientes métodos:

- Definir un método que imprima en pantalla los valores de los atributos del objeto.
- En un método *main* se deben crear dos personas y mostrar los va lores de sus atributos en pantalla

```
class Persona:
    def __init__(self, nombre, apellido,
numero_documento,año_nacimiento,pais_nacimiento,genero):
        self.nombre = nombre
        self.apellido = apellido
        self.numero_documento = numero_documento
        self.año_nacimiento = año_nacimiento
        self.pais_nacimiento = pais_nacimiento
        self.genero = genero

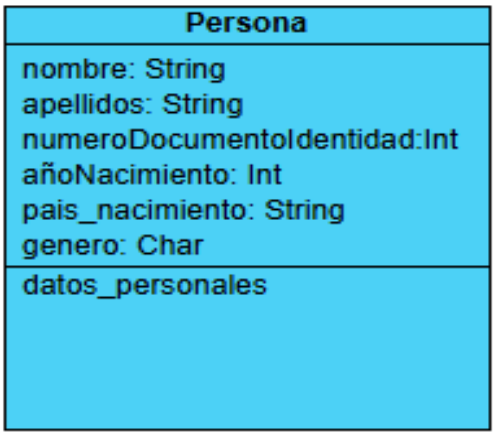
    def mostrar_datos_personales(self):
        print(f"Nombre = {self.nombre}")
        print(f"Apellido = {self.apellido}")
        print(f"Número de documento de identidad = {self.numero_documento}")
        print(f"Año de nacimiento = {self.año_nacimiento}")
        print(f"País de Nacimiento = {self.pais_nacimiento}")
        print(f"Género = {self.genero}")
        print()


nombre = input("Ingrese el nombre:\n")
apellido = input("Ingrese el apellido:\n")
numero_documento = input("Ingrese el número de documento de identidad:\n")
año_nacimiento = int(input("Ingrese el año de nacimiento:\n"))
pais_nacimiento = input("Ingrese su país de nacimiento:\n")
genero = input("Ingrese su género(H/M):\n")
```

```
persona1 = Persona(nombre, apellido, numero_documento,
año_nacimiento,pais_nacimiento,genero)
persona1.mostrar_datos_personales()

nombre = input("Ingrese el nombre:\n")
apellido = input("Ingrese el apellido:\n")
numero_documento = input("Ingrese el numero de documento de identidad:\n")
año_nacimiento = int(input("Ingrese el año de nacimiento:\n"))
pais_nacimiento = input("Ingrese su país de nacimiento:\n")
genero = input("Ingrese su género(H/M):\n")

persona2 = Persona(nombre, apellido, numero_documento,
año_nacimiento,pais_nacimiento,genero)
persona2.mostrar_datos_personales()
```



Ejercicio 2.2 (PAG 66)

Se requiere un programa que modele el concepto de un planeta del sistema solar. Un planeta tiene los siguientes atributos: u

- Un nombre de tipo *Sring* con valor inicial de *null*.
- Cantidad de satélites de tipo *int* con valor inicial de cero.
- Masa en kilogramos de tipo *double* con valor inicial de cero.
- Volumen en kilómetros cúbicos de tipo *double* con valor inicial de cero.
- Diámetro en kilómetros de tipo *int* con valor inicial de cero.
- Distancia media al Sol en millones de kilómetros, de tipo *int* con valor inicial de cero.
- Tipo de planeta de acuerdo con su tamaño, de tipo enumerado con los siguientes valores posibles: GASEOSO, TERRESTRE y ENANO.
- Observable a simple vista, de tipo booleano con valor inicial false. La clase debe incluir los siguientes métodos:

La clase debe tener un constructor que inicialice los valores de sus respectivos atributos.

- Definir un método que imprima en pantalla los valores de los atributos de un planeta.
- Calcular la densidad un planeta, como el cociente entre su masa y su volumen.
- Determinar si un planeta del sistema solar se considera exterior. Un planeta exterior está situado más allá del cinturón de asteroides. El cinturón de asteroides se encuentra entre 2.1 y 3.4 UA. Una unidad astronómica (UA) es la distancia entre la Tierra y el Sol= 149 597 870 Km.
- En un método *main* se deben crear dos planetas y mostrar los va lores de sus atributos en pantalla. Además, se debe imprimir la densidad de cada planeta y si el planeta es un planeta exterior del sistema solar

```python
from enum import Enum

class TipoPlaneta(Enum):
    GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"

class Planeta:
    def __init__(self, nombre, cantidad_satelites, masa, volumen, diametro,
distancia_sol, tipo, es_observable, perido_orbital, periodo_rotacion):
        self.nombre = nombre
        self.cantidad_satelites = cantidad_satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.es_observable = es_observable
        self.periodo_orbital = perido_orbital
        self.periodo_rotacion = periodo_rotacion

    def imprimir(self):
        print(f"Nombre del planeta = {self.nombre}")
        print(f"Cantidad de satélites = {self.cantidad_satelites}")
        print(f"Masa del planeta = {self.masa}")
        print(f"Volumen del planeta = {self.volumen}")
        print(f"Diámetro del planeta = {self.diametro}")
        print(f"Distancia al sol = {self.distancia_sol}")
        print(f"Tipo de planeta = {self.tipo.name}")
        print(f"Es observable = {self.es_observable}")
        print(f"Periodo Orbital en años = {self.periodo_orbital}")
        print(f"Periodo de Rotación en días = {self.periodo_rotacion}")

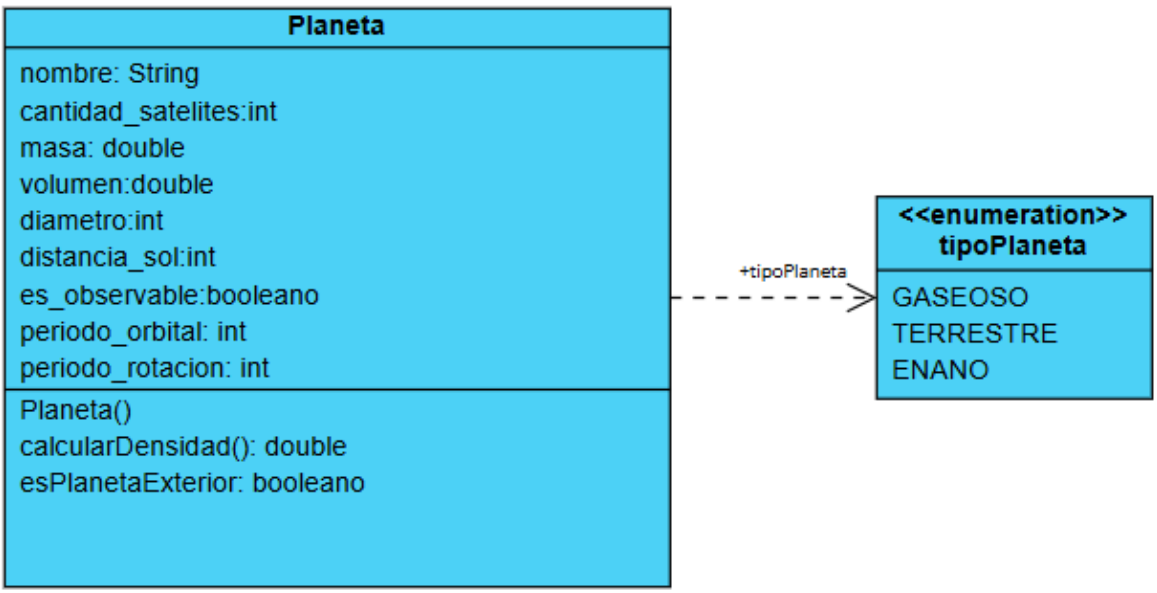

    def calcular_densidad(self):
        return self.masa / self.volumen if self.volumen != 0 else 0

    def es_planeta_exterior(self):
        limite = 508632758.0
        return self.distancia_sol > limite


p1 = Planeta(
    "Tierra", 1, 5.9736E24, 1.08321E12,
    12742, 150000000,TipoPlaneta.TERRESTRE,True, 1, 1);
```

```
print("**********************************")
p1.imprimir()
print(f"Densidad del planeta = {p1.calcular_densidad()}")
print(f"Es planeta exterior = {p1.es_planeta_exterior()}")
print("**********************************")

p2 = Planeta(
    "Júpiter", 79, 1.899E27, 1.4313E15,
    139820, 750000000,TipoPlaneta.GASEOSO,True, 11.86,0.41);
print("**********************************")
p2.imprimir()
print(f"Densidad del planeta = {p2.calcular_densidad()}")
print(f"Es planeta exterior = {p2.es_planeta_exterior()}")
print("**********************************")
```



Ejercicio 2.3 (PAG 66)

Se requiere un programa que modele el concepto de un automóvil. Un automóvil tiene los siguientes atributos:

- Marca: el nombre del fabricante.
- Modelo: año de fabricación.
- Motor: volumen en litros del cilindraje del motor de un automóvil.
- Tipo de combustible: valor enumerado con los posibles valores de gasolina, bioetanol, diésel, biodiésel, gas natural.
- Tipo de automóvil: valor enumerado con los posibles valores de carro de ciudad, subcompacto, compacto, familiar, ejecutivo, SUV.
- Número de puertas: cantidad de puertas.
- Cantidad de asientos: número de asientos disponibles que tiene el vehículo. u Velocidad máxima: velocidad máxima sostenida por el vehículo en km/h.
- Color: valor enumerado con los posibles valores de blanco, negro, rojo, naranja, amarillo, verde, azul, violeta.
- Velocidad actual: velocidad del vehículo en un momento dado.

La clase debe incluir los siguientes métodos:

- Un constructor para la clase Automóvil donde se le pasen como parámetros los valores de sus atributos.
- Métodos *get* y set para la clase Automóvil.
- Métodos para acelerar una cierta velocidad, desacelerar y frenar (colocar la velocidad actual en cero). Es importante tener en cuenta que no se debe acelerar más allá de la velocidad máxima permitida para el automóvil. De igual manera, tampoco es posible desacelerar a una velocidad negativa. Si se cumplen estos casos, se debe mostrar por pantalla los mensajes correspondientes.
- Un método para calcular el tiempo estimado de llegada, utilizando como parámetro la distancia a recorrer en kilómetros. El tiempo estimado se calcula como el cociente entre la distancia a recorrer y la velocidad actual.

- Un método para mostrar los valores de los atributos de un Auto móvil en pantalla.
- Un método *main* donde se deben crear un automóvil, colocar su velocidad actual en 100 km/h, aumentar su velocidad en 20 km/h, luego decrementar su velocidad en 50 km/h, y después frenar. Con cada cambio de velocidad, se debe mostrar en pantalla la velocidad actual.

```
from enum import Enum

class TipoA(Enum):
    CIUDAD = "CIUDAD"
    SUBCOMPACTO = "SUBCOMPACTO"
    COMPACTO = "COMPACTO"
    FAMILIAR = "FAMILIAR"
    EJECUTIVO = "EJECUTIVO"
    SUV = "SUV"

class TipoColor(Enum):
    BLANCO = "BLANCO"
    NEGRO = "NEGRO"
    ROJO = "ROJO"
    NARANJA = "NARANJA"
    AMARILLO = "AMARILLO"
    VERDE = "VERDE"
    AZUL = "AZUL"
    VIOLETA = "VIOLETA"

class TipoCom(Enum):
    GASOLINA = "GASOLINA"
    BIOETANOL = "BIOETANOL"
    DIESEL = "DIESEL"
    BIODIESEL = "BIODIESEL"
    GAS_NATURAL = "GAS_NATURAL"

class Automovil:
    def __init__(self, marca: str, modelo: int, motor: int,
                 tipo_combustible: TipoCom, tipo_automovil: TipoA,
                 numero_puertas: int, cantidad_asientos: int,
                 velocidad_maxima: int, color: TipoColor,
                 automatico: bool):

        self.marca = marca
```

```
        self.modelo = modelo
        self.motor = motor
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
        self.numero_puertas = numero_puertas
        self.cantidad_asientos = cantidad_asientos
        self.velocidad_maxima = velocidad_maxima
        self.color = color
        self.velocidad_actual = 0

        self.automatico = automatico

        self.cantidad_multas = 0
        self.valor_multa = 100000
        self.total_multas = 0

    def get_automatico(self) -> bool:
        return self.automatico

    def set_automatico(self, automatico: bool):
        self.automatico = automatico

    def acelerar(self, incremento_velocidad: int):
        if incremento_velocidad < 0:
            print("El incremento de velocidad no puede ser negativo")
        elif self.velocidad_actual + incremento_velocidad <=
self.velocidad_maxima:
            self.velocidad_actual += incremento_velocidad
        else:
            print("No se puede superar la velocidad máxima del automóvil")

            self.cantidad_multas += 1
            self.total_multas += self.valor_multa

            print(f"Multa generada: ${self.valor_multa}")

    def desacelerar(self, decremento_velocidad: int):
        if (self.velocidad_actual - decremento_velocidad) >= 0:
            self.velocidad_actual -= decremento_velocidad
        else:
            print("No se puede decrementar a una velocidad negativa")

    def frenar(self):
        self.velocidad_actual = 0
```

```python
    def calcular_tiempo_llegada(self, distancia: int) -> float:
        if self.velocidad_actual == 0:
            return float('inf')
        return distancia / self.velocidad_actual

    def tiene_multas(self) -> bool:
        return self.cantidad_multas > 0

    def calcular_total_multas(self) -> int:
        return self.total_multas

    def imprimir(self):
        print(f"Marca = {self.marca}")
        print(f"Modelo = {self.modelo}")
        print(f"Motor = {self.motor}")
        print(f"Tipo de combustible = {self.tipo_combustible.value}")
        print(f"Tipo de automóvil = {self.tipo_automovil.value}")
        print(f"Número de puertas = {self.numero_puertas}")
        print(f"Cantidad de asientos = {self.cantidad_asientos}")
        print(f"Velocidad máxima = {self.velocidad_maxima}")
        print(f"Color = {self.color.value}")
        print(f"¿Es automático? = {self.get_automatico()}")

if __name__ == "__main__":

    auto1 = Automovil(
        "Ford", 2018, 3,
        TipoCom.DIESEL,
        TipoA.EJECUTIVO,
        5, 6, 250,
        TipoColor.NEGRO,
        True
    )
    auto1.imprimir()
    auto1.velocidad_actual = 240
    auto1.acelerar(40)

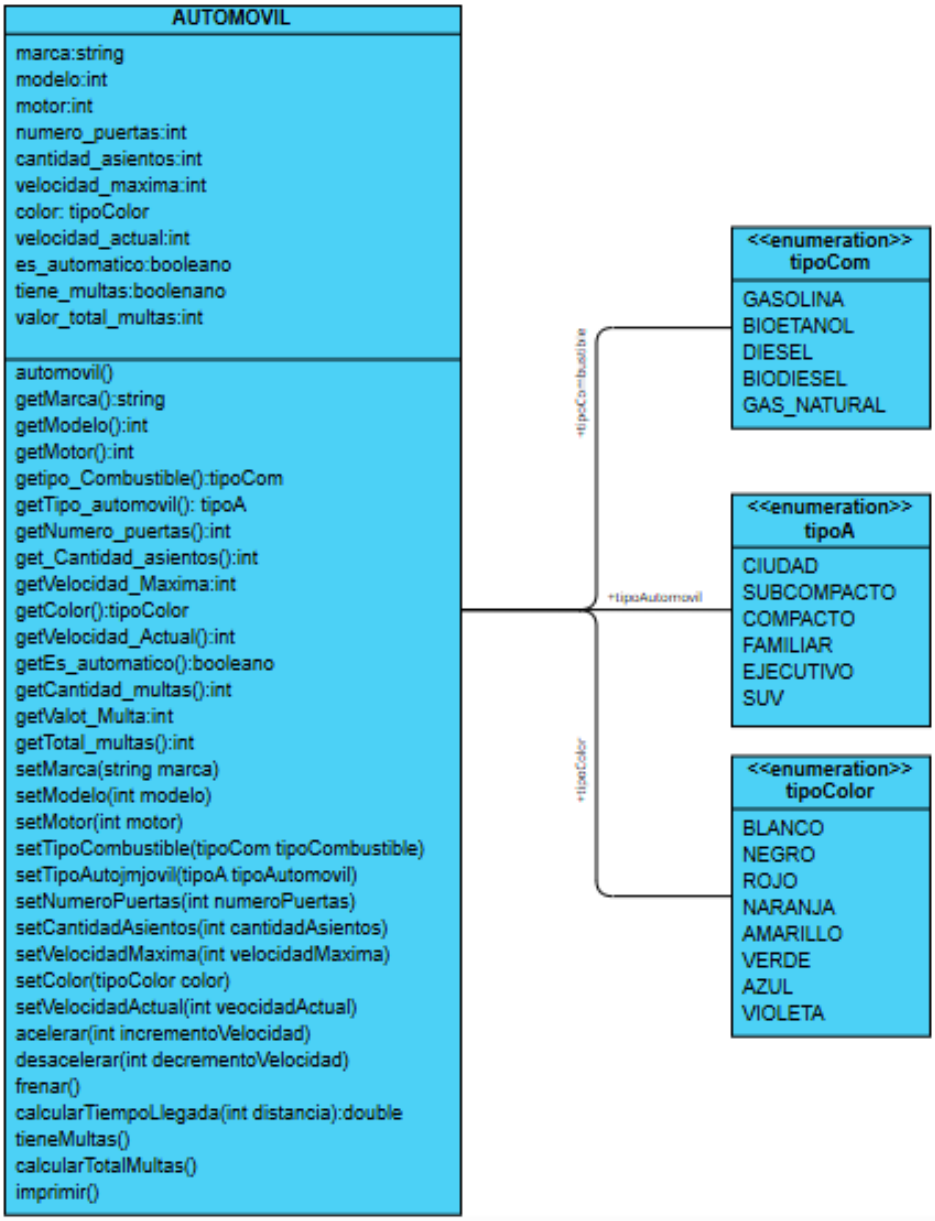
    print(f"\nVelocidad actual = {auto1.velocidad_actual}")
    print(f"Cantidad de multas = {auto1.cantidad_multas}")
    print(f"Valor Total de multas = ${auto1.calcular_total_multas()}")

    auto1.acelerar(20)
    print(f"Cantidad de multas = {auto1.cantidad_multas}")
    print(f"Valor Total de multas = ${auto1.calcular_total_multas()}")
```

```python
auto1.desacelerar(50)
print(f"Velocidad actual = {auto1.velocidad_actual}")

auto1.frenar()
print(f"Velocidad actual = {auto1.velocidad_actual}")

auto1.desacelerar(20)
```



Ejercicio 2.4 (PAG 86)

Se requiere un programa que modele varias figuras geométricas: el círculo, el rectángulo, el cuadrado y el triángulo rectángulo.

- El círculo tiene como atributo su radio en centímetros.
- El rectángulo, su base y altura en centímetros.
- El cuadrado, la longitud de sus lados en centímetros.
- El triángulo, su base y altura en centímetros.

Se requieren métodos para determinar el área y el perímetro de cada figura geométrica. Además, para el triángulo rectángulo se requiere:

- Un método que calcule la hipotenusa del rectángulo.
- Un método para determinar qué tipo de triángulo es:

Equilátero: todos sus lados son iguales.

Isósceles: tiene dos lados iguales.

Escaleno: todos sus lados son diferentes.

```python
import math

class Circulo:
    def __init__(self, radio: float):
        self.radio = radio

    def calcular_area(self):
        return math.pi * math.pow(self.radio, 2)

    def calcular_perimetro(self):
        return 2 * math.pi * self.radio

    def imprimir(self):
        print("CÍRCULO")
        print(f"Radio = {self.radio} cm")
        print(f"Área = {self.calcular_area():.2f} cm²")
        print(f"Perímetro = {self.calcular_perimetro():.2f} cm")
```

```python
class Rectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura

    def calcular_perimetro(self):
        return 2 * (self.base + self.altura)

    def imprimir(self):
        print("RECTÁNGULO")
        print(f"Base = {self.base} cm")
        print(f"Altura = {self.altura} cm")
        print(f"Área = {self.calcular_area():.2f} cm²")
        print(f"Perímetro = {self.calcular_perimetro():.2f} cm")


class Cuadrado:
    def __init__(self, lado: float):
        self.lado = lado

    def calcular_area(self):
        return math.pow(self.lado, 2)

    def calcular_perimetro(self):
        return 4 * self.lado

    def imprimir(self):
        print("CUADRADO")
        print(f"Lado = {self.lado} cm")
        print(f"Área = {self.calcular_area():.2f} cm²")
        print(f"Perímetro = {self.calcular_perimetro():.2f} cm")


class TrianguloRectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base
        self.altura = altura

    def calcular_hipotenusa(self):
        return math.sqrt(math.pow(self.base, 2) +
                         math.pow(self.altura, 2))
```

```
    def calcular_area(self):
        return (self.base * self.altura) / 2

    def calcular_perimetro(self):
        return (self.base + self.altura +
                self.calcular_hipotenusa())

    def determinar_tipo(self):
        hipotenusa = self.calcular_hipotenusa()

        if math.isclose(self.base, self.altura) and math.isclose(
                self.base, hipotenusa):
            return "Equilátero"
        elif math.isclose(self.base, self.altura) or math.isclose(
                self.base, hipotenusa) or math.isclose(
                self.altura, hipotenusa):
            return "Isósceles"
        else:
            return "Escaleno"

    def imprimir(self):
        print("TRIÁNGULO RECTÁNGULO")
        print(f"Base = {self.base} cm")
        print(f"Altura = {self.altura} cm")
        print(f"Hipotenusa = {self.calcular_hipotenusa():.2f} cm")
        print(f"Área = {self.calcular_area():.2f} cm²")
        print(f"Perímetro = {self.calcular_perimetro():.2f} cm")
        print(f"Tipo de triángulo = {self.determinar_tipo()}")

class Rombo:
    def __init__(self, diagonal_mayor: float, diagonal_menor: float):
        self.diagonal_mayor = diagonal_mayor
        self.diagonal_menor = diagonal_menor

    def calcular_area(self):
        return (self.diagonal_mayor * self.diagonal_menor) / 2

    def calcular_perimetro(self):
        lado = math.sqrt(
            math.pow(self.diagonal_mayor / 2, 2) +
            math.pow(self.diagonal_menor / 2, 2)
        )
        return 4 * lado

    def imprimir(self):
```

```
        print("ROMBO")
        print(f"Diagonal mayor = {self.diagonal_mayor} cm")
        print(f"Diagonal menor = {self.diagonal_menor} cm")
        print(f"El área del rombo es = {self.calcular_area()}")
        print(f"El perímetro del rombo es = {self.calcular_perimetro()}")

class Trapecio:
    def __init__(self, base_mayor: float, base_menor: float,
                 altura: float, lado1: float, lado2: float):
        self.base_mayor = base_mayor
        self.base_menor = base_menor
        self.altura = altura
        self.lado1 = lado1
        self.lado2 = lado2

    def calcular_area(self):
        return ((self.base_mayor + self.base_menor) * self.altura) / 2

    def calcular_perimetro(self):
        return (self.base_mayor + self.base_menor +
                self.lado1 + self.lado2)

    def imprimir(self):
        print("TRAPECIO")
        print(f"Base mayor = {self.base_mayor} cm")
        print(f"Base menor = {self.base_menor} cm")
        print(f"Altura = {self.altura} cm")
        print(f"Lado 1 = {self.lado1} cm")
        print(f"Lado 2 = {self.lado2} cm")
        print(f"El área del trapecio es = {self.calcular_area()}")
        print(f"El perímetro del trapecio es = {self.calcular_perimetro()}")

class PruebaFiguras:
    @staticmethod
    def main():

        circulo = Circulo(2)
        rectangulo = Rectangulo(1, 2)
        cuadrado = Cuadrado(3)
        triangulo = TrianguloRectangulo(3, 4)
        rombo = Rombo(9, 4)
        trapecio = Trapecio(10, 7, 4, 3, 3)
        print(f"El área del círculo es = {circulo.calcular_area()}")
        print(f"El perímetro del círculo es =
{circulo.calcular_perimetro()}")
```

```
        print()

        print(f"El área del rectángulo es = {rectangulo.calcular_area()}")
        print(f"El perímetro del rectángulo es =
{rectangulo.calcular_perimetro()}")
        print()

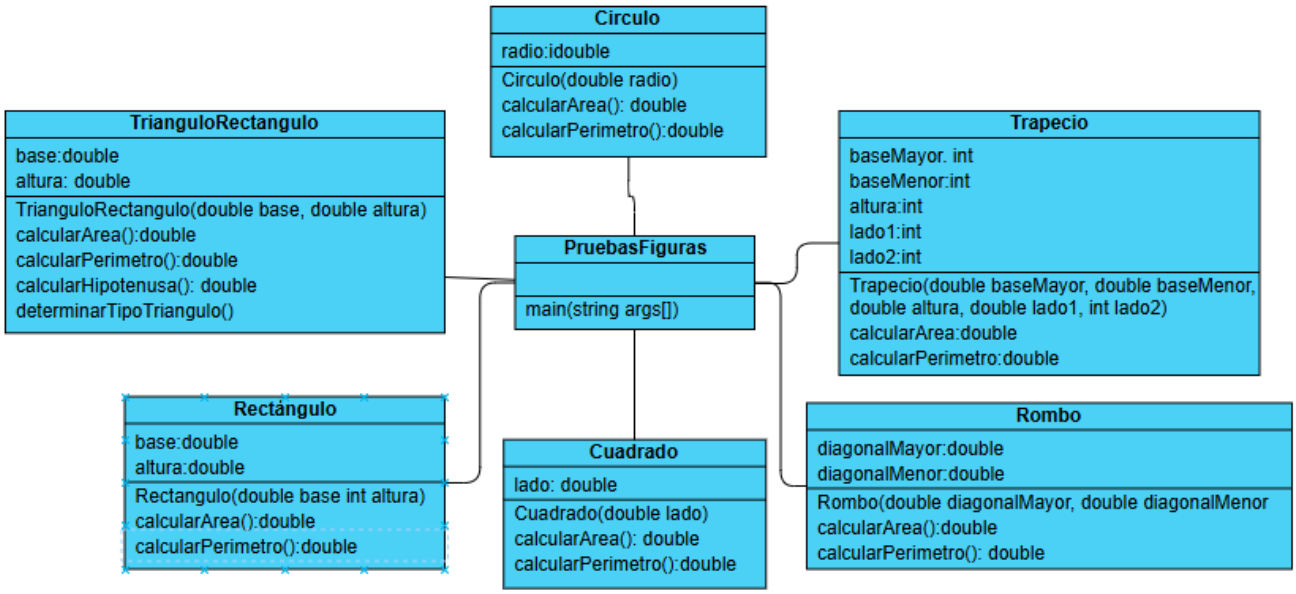
        print(f"El área del cuadrado es = {cuadrado.calcular_area()}")
        print(f"El perímetro del cuadrado es =
{cuadrado.calcular_perimetro()}")
        print()

        print(f"El área del triángulo es = {triangulo.calcular_area()}")
        print(f"El perímetro del triángulo es =
{triangulo.calcular_perimetro()}")
        print(f"Es un triángulo {triangulo.determinar_tipo()}")
        print()

        print(f"El área del rombo es = {rombo.calcular_area()}")
        print(f"El perímetro del rombo es = {rombo.calcular_perimetro()}")
        print()

        print(f"El área del trapecio es = {trapecio.calcular_area()}")
        print(f"El perímetro del trapecio es =
{trapecio.calcular_perimetro()}")

if __name__ == "__main__":
    PruebaFiguras.main()
```



Ejercicio 2.5 (PAG 95)

Se requiere un programa que modele una cuenta bancaria que posee los siguientes atributos:

- Nombres del titular.
- Apellidos del titular.
- Número de la cuenta bancaria.
- Tipo de cuenta: puede ser una cuenta de ahorros o una cuenta corriente.
- Saldo de la cuenta.

Se debe definir un constructor que inicialice los atributos de la clase. Cuando se crea una cuenta bancaria, su saldo inicial tiene un valor de cero. En una determinada cuenta bancaria se puede:

- Imprimir por pantalla los valores de los atributos de una cuenta bancaria. u Consultar el saldo de una cuenta bancaria.
- Consignar un determinado valor en la cuenta bancaria, actualizan do el saldo correspondiente.
- Retirar un determinado valor de la cuenta bancaria, actualizando el saldo correspondiente. Es necesario tener en cuenta que no se puede realizar el retiro si el valor solicitado supera el saldo actual de la cuenta

```
class CuentaBancaria:

    def __init__(self, nombres, apellidos, numero_cuenta, tipo_cuenta,
porcentaje_interes):
        self.nombres = nombres
        self.apellidos = apellidos
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0
        self.porcentaje_interes = porcentaje_interes

    def imprimir_datos(self):
        print("Nombres del titular =", self.nombres)
        print("Apellidos del titular =", self.apellidos)
        print("Número de cuenta =", self.numero_cuenta)
        print("Tipo de cuenta =", self.tipo_cuenta)
        print("Saldo =", self.saldo)

    def consignar(self, valor):
        self.saldo = self.saldo + valor
        print("Se ha consignado", valor, "en la cuenta. El nuevo saldo es",
self.saldo)

    def retirar(self, valor):
        if valor <= self.saldo:
            self.saldo = self.saldo - valor
            print("Se ha retirado", valor, "de la cuenta. El nuevo saldo
es", self.saldo)
        else:
            print("No se puede realizar el retiro. Saldo insuficiente.")

    def aplicar_interes(self):
        interes = self.saldo * self.porcentaje_interes / 100
        self.saldo = self.saldo + interes
        print("Se ha aplicado un interés de", self.porcentaje_interes, "%.")
        print("El nuevo saldo es", self.saldo)


cuenta = CuentaBancaria(
    "Pedro",
    "Pérez",
    "123456789",
    "AHORROS",
    2
```

```
)

cuenta.imprimir_datos()

cuenta.consignar(200000)

cuenta.consignar(300000)

cuenta.retirar(400000)

cuenta.aplicar_interes()
```

